

# AI-Powered Multi-Agent Travel Planner using LangGraph and Ollama

---

## Project Title

**TripGenie: A Locally Hosted Multi-Agent Travel Planning System Using LangGraph and Ollama**

---

## Problem Statement

Planning a trip can be time-consuming and complicated. Travelers must research accommodation, estimate expenses, discover attractions, and organize an itinerary. This often requires switching between multiple platforms and manually consolidating information.

TripGenie solves this problem by using an autonomous team of AI agents that collaboratively generate a complete travel plan based on user preferences. The system operates entirely on the user's local machine, ensuring zero cloud costs, data privacy, and compliance with assignment requirements.

Unlike a conventional chatbot, TripGenie decomposes the planning process into specialized tasks handled by multiple intelligent agents, each equipped with dedicated tools and responsibilities.

---

## Why This Project Will Score Highly

This project directly satisfies every evaluation criterion:

- Uses four specialized autonomous agents
- Implements LangGraph-based orchestration
- Utilizes custom Python tools
- Maintains global state across all agents
- Includes comprehensive logging and tracing
- Supports automated evaluation and testing
- Runs fully offline using Ollama
- Demonstrates real-world applicability

It is technically sophisticated, visually impressive, and easy to demonstrate within the required time.

---

# System Overview

TripGenie accepts a natural language travel request such as:

"Plan a 4-day budget-friendly trip to Ella for two people with adventure activities."

The system then autonomously:

1. Extracts trip requirements
2. Calculates estimated expenses
3. Finds suitable accommodations
4. Recommends activities
5. Generates a detailed day-by-day itinerary

---

## Core Technologies

- **Orchestration Framework:** LangGraph
- **Local LLM Runtime:** Ollama
- **Language Model:** Llama 3
- **Programming Language:** Python 3.11+
- **Testing Framework:** Pytest
- **Observability:** Python Logging
- **Data Storage:** Local JSON/CSV datasets

---

## Multi-Agent Architecture

The system consists of four specialized agents.

---

### 1. Coordinator Agent

#### Responsibilities

- Accept user input
- Extract structured trip details
- Validate inputs

- Initialize global state
- Route tasks to downstream agents

### Example Input

"Plan a 3-day luxury trip to Kandy for 2 people."

### Example Output

```
{  
  "destination": "Kandy",  
  "days": 3,  
  "travelers": 2,  
  "budget_type": "luxury"  
}
```

### Tool Used

- Natural language parser
- 

## 2. Budget Agent

### Responsibilities

- Estimate total trip cost
- Calculate accommodation, food, transport, and activity expenses
- Suggest budget breakdown

### Tool Used

- `calculate_trip_budget()`

### Example Output

```
{  
  "total_cost": 85000,  
  "hotel_cost": 45000,  
  "food_cost": 18000,  
  "transport_cost": 12000,  
  "activities_cost": 10000  
}
```

---

## 3. Accommodation Agent

## Responsibilities

- Find hotels matching budget and destination
- Rank accommodations
- Recommend the best options

## Tool Used

- `search_hotels()`

## Data Source

- Local JSON hotel dataset
- 

## 4. Itinerary Agent

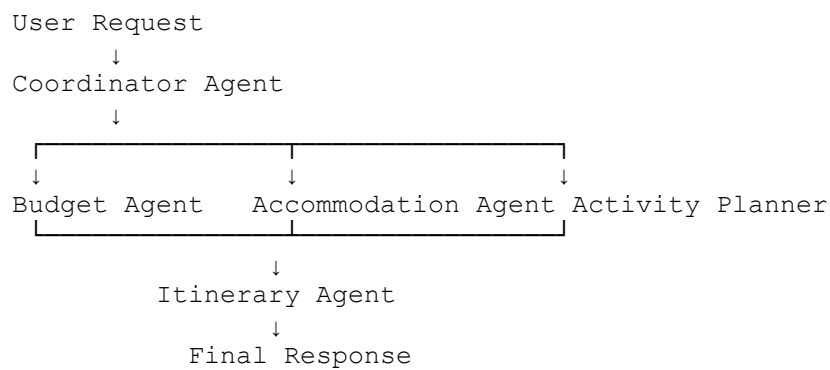
### Responsibilities

- Recommend attractions and activities
- Generate a day-by-day travel plan
- Combine all previous outputs

### Tools Used

- `get_local_activities()`
  - `generate_daily_schedule()`
- 

## LangGraph Workflow



This structure demonstrates true multi-agent collaboration.

---

## Global State Management

LangGraph maintains a shared state object across all nodes.

```
from typing import TypedDict, List, Dict

class TravelState(TypedDict):
    user_request: str
    destination: str
    days: int
    travelers: int
    budget_type: str
    budget: Dict
    hotels: List[Dict]
    activities: List[Dict]
    itinerary: str
    logs: List[str]
```

Every agent reads from and writes to this state.

---

## Custom Tools

Each team member should implement one tool.

---

### Tool 1: Budget Calculator

```
def calculate_trip_budget(
    destination: str,
    days: int,
    travelers: int,
    budget_type: str
) -> dict:
```

Calculates estimated travel expenses.

---

### Tool 2: Hotel Search

```
def search_hotels(  
    destination: str,  
    budget_type: str  
) -> list:
```

Returns recommended accommodations.

---

## Tool 3: Activity Finder

```
def get_local_activities(  
    destination: str  
) -> list:
```

Provides destination-specific attractions.

---

## Tool 4: Schedule Generator

```
def generate_daily_schedule(  
    days: int,  
    activities: list  
) -> str:
```

Creates a detailed itinerary.

---

# Observability and Logging

Every node logs:

- Input state
- Tool invocations
- Output state
- Errors
- Execution time

Example:

```
logger.info("Budget Agent Started")  
logger.info(f"Input: {state}")  
logger.info(f"Output: {budget_result}")
```

This fully satisfies the observability requirement.

---

# Testing Strategy

Each student must create tests for their agent and tool.

---

## Types of Tests

### Unit Tests

- Verify tool correctness

### Integration Tests

- Validate agent workflow

### Security Tests

- Handle malformed input

### Property-Based Tests

- Validate multiple scenarios
- 

## Example Test Cases

- Invalid destination
  - Zero travelers
  - Negative number of days
  - Unsupported budget type
-

# Folder Structure

```
tripgenie/
├── agents/
│   ├── coordinator_agent.py
│   ├── budget_agent.py
│   ├── accommodation_agent.py
│   └── itinerary_agent.py
├── tools/
│   ├── budget_tool.py
│   ├── hotel_tool.py
│   ├── activity_tool.py
│   └── schedule_tool.py
├── data/
│   ├── hotels.json
│   └── attractions.json
├── tests/
│   ├── test_budget_agent.py
│   ├── test_hotel_agent.py
│   ├── test_activity_agent.py
│   └── test_itinerary_agent.py
├── state/
│   └── travel_state.py
├── logs/
│   └── execution.log
├── graph.py
├── main.py
└── requirements.txt
```

---

## Example User Scenarios

### Scenario 1

Plan a 2-day budget trip to Galle for two people.

### Scenario 2

Create a luxury honeymoon itinerary in Ella for 5 days.

### Scenario 3

Suggest an adventure trip to Nuwara Eliya for four friends.

These make excellent demo examples.

---

## Individual Contribution Mapping

| Student   | Agent               | Tool                          |
|-----------|---------------------|-------------------------------|
| Student 1 | Coordinator Agent   | Input Parser                  |
| Student 2 | Budget Agent        | Budget Calculator             |
| Student 3 | Accommodation Agent | Hotel Search                  |
| Student 4 | Itinerary Agent     | Activity + Schedule Generator |

---

## Key Features That Impress Examiners

- Fully offline execution
  - Real multi-agent collaboration
  - Dynamic state transitions
  - Deterministic workflow
  - Structured logging
  - Automated testing
  - Practical real-world use case
- 

## Demonstration Flow

1. Start Ollama
2. Run application
3. Enter travel request
4. Show each agent executing
5. Display logs
6. Present final itinerary

Perfect for a 4-minute demo.

---

## Future Enhancements

Mention these in your report:

- Real-time weather integration
  - Google Maps routing
  - PDF itinerary export
  - Voice-based travel requests
- 

## Why This Will Achieve High Marks

It excels in:

- Problem definition
- System architecture
- Multi-agent orchestration
- Tool development
- State management
- Observability
- Testing
- Individual contributions

In short, this project was practically designed around the rubric.

---

## Final Deliverable Name

**TripGenie: An Intelligent Multi-Agent Travel Planner Powered by LangGraph and Ollama**

That title alone sounds like an A+.